Lec-4

31/05/26 12:43 PM

(i) Number System (ii) Data Type (iii) Type Casting

(i) Binary number System → 2 → [0, 1]
(ii) decimal number System → 10 → [0, 9]
(iii) octal number System → 8 → [0, 7]
→ 5 → [0, 4]

decimal

$378 = 3 \times 10^2 + 7 \times 10^1 + 8 \times 10^0$

  3784
+ 2687
= 6471

  2345
+ 4321
= 1110

6 % 6 = 0, 6/6 = 1, 7 % 6 = 1, 7/6 = 1

11 % 10 = 1, 11/10 = 1, 6 % 10 = 6, 6/10 = 0

17 % 10 = 7, 17/10 = 1, 14 % 10 = 4, 14/10 = 1

  01101
+ 101101
  001101
= 111010

2 % 2 = 0, 2/2 = 1, 1 % 2 = 1, 1/2 = 0, 3 % 2 = 1, 3/2 = 1

2	39
2	19
2	9
2	4
2	2
2	1
	0

1×10^0, 1×10^1, 1×10^2, 0×10^3, 0×10^4, 1×10^5

100111 → $100111 = 1 \times 10^5 + 0 \times 10^4 + 0 \times 10^3 + 1 \times 10^2 + 1 \times 10^1 + 1 \times 10^0$

while (n > 0) {
rem = n % 2
sum = sum + rem × mul
n = n / 2
mul = mul × 10
}

sum: $1 \times 10^0 + 1 \times 10^1 + 1 \times 10^2 + 0 + 0 + 10^5$

mul = 1, 10, 100, 10^3, 10^4, 10^5

10	100101
10	10010
10	1001
10	100
10	10
10	1
	0

1×2^0, 0×2^1, 1×2^2, 0×2^3, 0×2^4, 1×2^5

1 + 4 + 32 = 37

while (n > 0) {
rem = n % 10
sum = sum + rem × mul
n = n / 10
mul = mul × 2
}

```java
public static void main(String[] args) {
        // TODO Auto-generated method stub
        int mul = 1;
        int sum = 0;
        while (n > 0) {
            int rem = n % des;
            sum = sum + rem * mul;
            n = n / des;
            mul = mul * src;
        }
        System.out.println(sum);

    }
```

src 8, 10

(3214) 8 → 9, 10, new num

Data Types

primitive / non primitive

byte b = 5

- byte → 8 bit → 1 byte → Range
- short → 16 bit → 2 byte
- int → 32 bit → 4 byte
- long → 64 bit → 8 byte
- float → floating → 32 bit → 4 byte
- double → 64 bit → 8 byte
- char → char → 2 byte → 16 bit
- boolean → True / False → 1 bit — 8 bit

Sign bit: −ve 1, +ve 0 (M.S.B)

$2^8 = 256$

[−127, 127] → 255

1 000 0000 → −0
0 000 0000 → +0

1's → compliment

00010100 → x
11101011 → −x

2's → compliment

01101101 → x
10010010
00000001
10010011 → −2

(−5) −86K

−ve 1 0 0 0 0 0 0 0
+ve 0 1 1 1 1 1 1 1

[−128, 127]

1000 0000
01111111
0000 0001
10000000

2 % 2 = 0, 2/2 = 1

byte	8 bit	−128 to 127	-2^7 to $2^7 - 1$
short	16 bit		-2^{15} to $2^{15} - 1$
int	32 bit		-2^{31} to $2^{31} - 1$
long	64 bit		-2^{63} to $2^{63} - 1$

$2^7 \ldots 2^4\ 2^3\ 2^2\ 2^1\ 2^0$

-2^{31} to $2^{31} - 1$ → 10^9

-2^{63} to $2^{63} - 1$ → 10^{18} → long, long

byte b = 5

0000 0101 ✓

byte b = −5

0000 0101
1111 1010
1
1111 1011 → −5

−ve

```java
byte b = (byte)(428);
```

1 10101100

10101100
01010011
1
01010100 → −84

```java
byte b = (byte)(300);
```

1 00101100

00101100 → +44

00000

32 zero → 24 zero → 00000101 int i = 5

11111011

P, B

16 → 2^{16}

char → ASCII → Unicode

256

97 ← a	65 A	'0' 48
98 ← b	66 B	'1' 49
99 ← c	67 C	'2' 50
		51
z	Z	'9'

26 → A to Z

S2 → 5, 21

```java
for (byte i = 0; i < 128; i++) {
        System.out.println(i);
    }
```

i = byte (i+1), byte (127+1) → −128

0, 1, …, 125, 126, 127, −128